

Symptom-Based Disease Prediction & Possible-Condition Analysis

A step-by-step Machine Learning project guide for an MSc Artificial Intelligence student

Goal: Build an educational ML prototype that takes a set of symptoms, predicts possible condition classes, and displays the model's top predictions with estimated probabilities. The system should never present itself as a medical diagnosis tool.

Recommended starting point: tabular multiclass classification using symptom features. Start with Logistic Regression, then compare Decision Tree and Random Forest before adding explainability and a Streamlit interface.

Important: Health-related AI can produce unsafe or misleading outputs when data are poor, biased, non-representative, or used without appropriate oversight. Treat this as an educational/technical project and clearly communicate limitations.

1. The Whole Project Idea

The system receives symptoms as input and uses a trained classification model to estimate which condition classes are most compatible with the symptom pattern in the training data.

Basic flow:

```
USER
↓
Select / enter symptoms
↓
Convert symptoms to numerical features
↓
Machine-learning model
↓
Probability estimate for each class
↓
Top 3 possible conditions
↓
Educational explanation + safety notice
```

Example:

```
Input:
✓ Fever
✓ Cough
✓ Fatigue
✗ Skin rash
✗ Joint pain

Output:
1. Condition A — 72%
2. Condition B — 16%
3. Condition C — 8%

Interpretation:
These are model-generated possibilities, not a diagnosis.
```

2. What Type of Machine Learning Problem Is This?

This is primarily a **multiclass classification** problem. The input is a collection of symptom features and the target is one class from several possible classes.

```
X = symptoms/features
y = condition/disease class

X → ML model → y
```

If there are 30 possible classes, the model learns to distinguish among those 30 classes. The important first step is to understand exactly what the dataset labels mean before training anything.

3. Step 1 — Choose and Inspect the Dataset

Your first practical task is to find a suitable symptom-condition dataset for educational use. Do not assume that a public dataset is clinically representative simply because it contains disease names and symptoms.

Before training, answer:

- How many rows are there?
- How many symptom features?
- What is the target column?
- How many condition classes?
- Are there missing values?
- Are there duplicate rows?

- How many examples belong to each class?
- How were the labels and symptoms collected?

```
import pandas as pd

df = pd.read_csv("disease_dataset.csv")

print(df.head())
print(df.shape)
print(df.columns)
print(df.info())
print(df.isnull().sum())
print(df["disease"].value_counts())
```

Expected understanding: **data first, model second**. Do not start by training several algorithms without understanding the dataset.

4. Step 2 — Understand X and y

Suppose your table looks like this:

fever	cough	headache	fatigue	disease
1	1	0	1	Condition A
0	1	1	0	Condition B
1	0	1	1	Condition C

X contains the symptom features. **y** contains the target class.

```
x = df.drop("disease", axis=1)
y = df["disease"]
```

Think of it as: **X = what the model sees; y = what the model must learn to predict**.

5. Step 3 — Represent Symptoms Numerically

Most symptom datasets use binary features: 1 means present and 0 means absent.

```
Patient:
fever = 1
cough = 1
headache = 0
fatigue = 1

Feature vector:
[1, 1, 0, 1]
```

This feature-vector idea connects directly to the mathematics you are studying: a patient's symptoms become a numerical vector that the ML algorithm can process.

6. Step 4 — Split Training and Test Data

Do not train and evaluate on exactly the same examples. A common starting split is 80% training and 20% testing.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Training set: used to learn patterns.

Test set: held back to estimate performance on unseen examples.

stratify=y: helps preserve class proportions across the split.

For a serious study, later use cross-validation and a carefully defined validation strategy rather than repeatedly tuning on the test set.

7. Step 5 — Train the First Model: Logistic Regression

Start simple. Logistic Regression is a useful baseline for multiclass classification and can produce class probability estimates.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

The `fit()` step is where the model learns relationships between symptom features and target classes.

8. Step 6 — Make Predictions

```
predictions = model.predict(X_test)
print(predictions)
```

The output is a predicted class for each test example.

9. Step 7 — Get Probability Estimates

```
probabilities = model.predict_proba(X_test)
print(probabilities[0])
```

For one example, you may see something conceptually like:

```
Condition A → 0.72
Condition B → 0.16
Condition C → 0.08
Condition D → 0.04
```

These are **model probability estimates**. They should not automatically be interpreted as medically validated probabilities or as a person's actual chance of having a disease.

10. Step 8 — Return the Top 3 Classes

Instead of showing only one answer, return several high-scoring classes. This better communicates uncertainty and is useful for an educational differential-condition prototype.

```
import numpy as np

probs = model.predict_proba(X_test[0:1])[0]
classes = model.classes_

top_indices = np.argsort(probs)[::-1][:3]

for i in top_indices:
    print(classes[i], probs[i])
```

Conceptually:

```
Top 3:
1. Condition A — 72%
2. Condition B — 16%
3. Condition C — 8%
```

11. Step 9 — Understand max and argmax

This connects directly to the maths you have been studying.

```
Probabilities = [0.10, 0.70, 0.20]

max(probabilities) → 0.70
argmax(probabilities) → position of 0.70
```

max asks: what is the largest value?

argmax asks: which class has the largest value?

12. Step 10 — Evaluate the Model

Do not report only accuracy. A health-related classifier can look good on accuracy while performing poorly for particular classes.

```
from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

Look at **precision**, **recall**, and **F1-score** for each class. Interpret them in the context of class frequency and the intended use.

13. Step 11 — Confusion Matrix

A confusion matrix shows which classes are being confused with each other.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, predictions)
print(cm)
```

For example, if Condition B is frequently predicted as Condition C, you should investigate whether the classes have overlapping symptom patterns or whether the dataset is insufficient to separate them.

14. Step 12 — Check Class Imbalance

```
print(y.value_counts())
print(y.value_counts(normalize=True))
```

Example: if one class has 1,000 records and another has only 20, the model has much more training information for the first class. Class imbalance can distort simple performance measures.

Later you can investigate class weighting, resampling, macro-averaged metrics, and per-class performance. Do not apply a balancing method blindly; document why you used it.

15. Step 13 — Avoid Data Leakage

Data leakage occurs when information that should be unavailable during training influences the training process or preprocessing. This can make test performance look artificially good.

```
Correct high-level workflow:

Raw data
↓
Train/test split
↓
Fit preprocessing using training data
↓
Train model
↓
Transform/evaluate held-out test data
```

When preprocessing becomes more complex, use a scikit-learn Pipeline so that transformations are fitted in the correct place.

16. Step 14 — Compare Several Models

After your baseline works, compare a small set of classical models.

Model	Why use it?
Logistic Regression	Strong, simple baseline; relatively interpretable.
Decision Tree	Easy to visualize and explain.
Random Forest	Ensemble of trees; useful nonlinear baseline.
Gradient Boosting	Later experiment for a stronger tabular baseline.

Compare models using the same evaluation strategy. Do not select a model using an arbitrary single metric.

17. Step 15 — Add Explainability

Once the core model works, investigate why it is making predictions. Useful techniques include feature importance, permutation importance, and later SHAP.

For example, an explanation layer might show which symptoms were most influential for a particular model prediction. The explanation itself should also be treated cautiously: an explanation method is not proof of clinical causation.

18. Step 16 — Build the User Interface

For your first version, Streamlit is a practical choice because it lets you create a small ML application directly from Python.

Suggested interface:

```
Symptom Analysis Assistant

Select symptoms:
■ Fever
■ Cough
■ Fatigue
■ Headache
■ Joint pain
■ Skin rash

[ Analyse ]

Model-generated possible conditions:
1. Condition A — 72%
2. Condition B — 16%
3. Condition C — 8%

Educational prototype only.
Not a medical diagnosis.
```

19. Final Technical Architecture

```
USER
■
↓
Streamlit UI
■
↓
Input validation
■
↓
Symptom preprocessing
■
↓
Feature vector
```


9. Check missing values and duplicates.
10. Check the number of examples in every target class.

```
df.head()
df.shape
df.columns
df.info()
df.isnull().sum()
df.duplicated().sum()
df["disease"].value_counts()
```

Evening:

11. Create X and y.
12. Split the dataset.
13. Train Logistic Regression.
14. Predict on the test set.
15. Print a classification report.

```
X = df.drop("disease", axis=1)
y = df["disease"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(classification_report(y_test, predictions))
```

If you can do these 15 tasks, **Day 1 is complete.**

22. Two-Week Roadmap

Day	Goal
1	Dataset, exploration, first Logistic Regression model
2	Preprocessing, validation, leakage prevention
3	Understand and tune Logistic Regression
4	Decision Tree
5	Random Forest
6	Model comparison
7	Confusion matrix and detailed evaluation
8	Top-3 predictions and probability output
9	Probability/calibration analysis
10	Feature importance / explainability
11	Streamlit interface
12	Connect UI to trained model
13	Testing and edge cases
14	README, architecture, limitations, final documentation

23. How to Make It More MSc-Level

Once the basic prototype works, investigate questions rather than just adding features:

- How does performance change across different classes?
- Which symptoms are most informative?
- How robust is the model when some symptoms are missing?
- Are probability estimates calibrated?
- Does class imbalance affect performance?
- Which conditions are commonly confused?
- How does performance change between models?
- What are the limitations of the dataset?
- What kinds of errors would be especially concerning in a health setting?

This turns the project from a simple 'train a classifier' exercise into an investigation of **reliability, uncertainty, data quality, and responsible AI**.

24. Version 2 — Natural-Language Symptoms

After the structured symptom-input version works, you can allow a user to type a sentence such as:

```
"I have fever, headache and fatigue."
```

Then a separate symptom-extraction component can map the text to the structured feature vector:

```
fever = 1
headache = 1
fatigue = 1
other symptoms = 0
```

Only after the underlying ML pipeline is reliable should you consider adding an LLM or more advanced NLP layer. Do not make the LLM the foundation of the first version.

25. Safety and Scope

Use a clear statement in the application and README:

Educational prototype only. The system provides model-generated possible condition classes from the supplied symptoms. It is not a medical diagnosis, does not replace a qualified healthcare professional, and should not be used to make medical decisions.

Avoid claims such as 'the model knows what disease you have' or '95% accurate means you can trust the diagnosis.' Accuracy on a particular dataset does not establish clinical validity.

26. Final Checklist

Before calling the project complete, you should be able to demonstrate:

- Dataset source and limitations documented
- X and y clearly defined
- Train/test strategy documented
- Missing values and duplicates checked
- Class distribution checked
- Baseline model trained
- At least two additional models compared
- Precision, recall and F1 reported
- Confusion matrix analysed
- Probability outputs handled carefully
- Top-3 predictions implemented
- Explainability investigated
- Streamlit interface implemented

- Error cases tested
- Limitations documented
- Medical/safety disclaimer included
- README contains setup and usage instructions

27. Your First Target

Tomorrow, do only this:

```
DATASET
↓
Explore it
↓
Identify X and y
↓
Train/test split
↓
Logistic Regression
↓
Prediction
↓
Classification report
```

Once this works, the next step is not to add more complexity immediately. The next step is to understand every line of the pipeline and then improve it carefully.

Recommended working style: build one small component, run it, understand the output, and only then move to the next component. This will teach you much more than copying a complete project from the internet.